

主题：Android 系统中联系人、短信、彩信、相册图片的获取与操作（包括 Java + XML 实现，权限申请及核心逻辑）

Android 系统操作详解：联系人、短信、彩信、相册图片获取（Java + XML）

在 Android 开发中，访问联系人、短信、彩信或从相册获取图片是常见的系统交互场景，这些操作均涉及**权限申请**和**系统 API 调用**。本文将分模块详细讲解每个功能的实现步骤、权限处理及核心逻辑（基于 Java + XML）。

一、访问手机联系人

1. 权限申请

访问联系人需要**危险权限**，需在清单文件声明并动态申请：

Code block

```
1  <!-- 读取联系人权限 -->
2  <uses-permission android:name="android.permission.READ_CONTACTS" />
3  <!-- 写入联系人权限（如需修改/添加联系人） -->
4  <uses-permission android:name="android.permission.WRITE_CONTACTS" />
```

2. 核心 API：ContentResolver

Android 联系人数据通过**内容提供者（ContentProvider）**暴露，需用 `ContentResolver` 查询 `ContactsContract` 相关表。

3. 实战实现（读取联系人列表）

步骤 1：XML 布局（显示联系人列表）

Code block

```
1  <!-- activity_contacts.xml -->
2  <?xml version="1.0" encoding="utf-8"?>
3  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
```

```

5      android:layout_height="match_parent"
6      android:orientation="vertical">
7
8      <Button
9          android:id="@+id/btn_read_contacts"
10         android:layout_width="wrap_content"
11         android:layout_height="wrap_content"
12         android:text="读取联系人" />
13
14     <ListView
15         android:id="@+id/lv_contacts"
16         android:layout_width="match_parent"
17         android:layout_height="match_parent" />
18
19 </LinearLayout>

```

步骤 2: Java 逻辑 (权限申请 + 读取联系人)

Code block

```

1  import android.Manifest;
2  import android.content.pm.PackageManager;
3  import android.database.Cursor;
4  import android.os.Bundle;
5  import android.provider.ContactsContract;
6  import android.view.View;
7  import android.widget.AdapterView;
8  import android.widget.Button;
9  import android.widget.ListView;
10 import android.widget.Toast;
11
12 import androidx.annotation.NonNull;
13 import androidx.appcompat.app.AppCompatActivity;
14 import androidx.core.app.ActivityCompat;
15 import androidx.core.content.ContextCompat;
16
17 import java.util.ArrayList;
18 import java.util.List;
19
20 public class ContactsActivity extends AppCompatActivity {
21
22     private static final int REQUEST_READ_CONTACTS = 2001;
23     private Button btnReadContacts;
24     private ListView lvContacts;
25     private List<String> contactList = new ArrayList<>();
26     private ArrayAdapter<String> adapter;

```

```

27
28     @Override
29     protected void onCreate(Bundle savedInstanceState) {
30         super.onCreate(savedInstanceState);
31         setContentView(R.layout.activity_contacts);
32
33         btnReadContacts = findViewById(R.id.btn_read_contacts);
34         lvContacts = findViewById(R.id.lv_contacts);
35         adapter = new ArrayAdapter<>(this,
android.R.layout.simple_list_item_1, contactList);
36         lvContacts.setAdapter(adapter);
37
38         btnReadContacts.setOnClickListener(v -> checkContactsPermission());
39     }
40
41     // 检查读取联系人权限
42     private void checkContactsPermission() {
43         if (ContextCompat.checkSelfPermission(this,
Manifest.permission.READ_CONTACTS) != PackageManager.PERMISSION_GRANTED) {
44             ActivityCompat.requestPermissions(this, new String[]
{Manifest.permission.READ_CONTACTS}, REQUEST_READ_CONTACTS);
45         } else {
46             readContacts(); // 权限已授予，读取联系人
47         }
48     }
49
50     // 读取联系人核心逻辑
51     private void readContacts() {
52         contactList.clear();
53         // 查询联系人数据 (ContentResolver)
54         Cursor cursor = getContentResolver().query(
55             ContactsContract.CommonDataKinds.Phone.CONTENT_URI,
56             null, null, null, null
57         );
58
59         if (cursor != null) {
60             while (cursor.moveToNext()) {
61                 // 获取联系人姓名和手机号
62                 String name =
cursor.getString(cursor.getColumnIndex(ContactsContract.CommonDataKinds.Phone.D
ISPLAY_NAME));
63                 String phone =
cursor.getString(cursor.getColumnIndex(ContactsContract.CommonDataKinds.Phone.N
UMBER));
64                 contactList.add(name + "\n" + phone);
65             }
66             adapter.notifyDataSetChanged();

```

```

67         cursor.close();
68     }
69 }
70
71 // 处理权限申请结果
72 @Override
73 public void onRequestPermissionsResult(int requestCode, @NonNull String[]
permissions, @NonNull int[] grantResults) {
74     super.onRequestPermissionsResult(requestCode, permissions,
grantResults);
75     if (requestCode == REQUEST_READ_CONTACTS) {
76         if (grantResults.length > 0 && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
77             readContacts();
78         }
79     }
80 }

```

4. 关键细节

- **ContactsContract 表结构**：联系人姓名存在 `DISPLAY_NAME` 列，手机号存在 `NUMBER` 列；
- **Cursor 关闭**：查询完成后必须关闭 Cursor，避免内存泄漏；
- **安卓 10+ 适配**：读取联系人无需额外适配，权限逻辑不变。

二、读取/发送短信

1. 权限申请

短信操作涉及多个危险权限，需声明并动态申请：

Code block

```

1  <!-- 读取短信 -->
2  <uses-permission android:name="android.permission.READ_SMS" />
3  <!-- 发送短信 -->
4  <uses-permission android:name="android.permission.SEND_SMS" />
5  <!-- 接收短信（如需监听） -->
6  <uses-permission android:name="android.permission.RECEIVE_SMS" />

```

2. 核心 API

- **读取短信**：通过 `ContentResolver` 查询 `Telephony.Sms.CONTENT_URI`；
- **发送短信**：使用 `SmsManager` 类。

3. 实战实现（读取短信 + 发送短信）

步骤 1：XML 布局

```

1</code></block>activity_sms.xml -->
2  <?xml version="1.0" encoding="utf-8"?>
3  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:orientation="vertical"
7      android:padding="16dp">
8
9      <Button
10         android:id="@+id/btn_read_sms"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="读取短信" />
14
15      <ListView
16         android:id="@+id/lv_sms"
17         android:layout_width="match_parent"
18         android:layout_height="0dp"
19         android:layout_weight="1" />
20
21      <EditText
22         android:id="@+id/et_phone"
23         android:layout_width="match_parent"
24         android:layout_height="wrap_content"
25         android:hint="输入手机号" />
26
27      <EditText
28         android:id="@+id/et_message"
29         android:layout_width="match_parent"
30         android:layout_height="wrap_content"
31         android:hint="输入短信内容" />
32
33      <Button
34         android:id="@+id/btn_send_sms"
35         android:layout_width="wrap_content"
36         android:layout_height="wrap_content"
37         android:text="发送短信" />
38
39  </LinearLayout>

```

步骤 2: Java 逻辑

Code block

```

1  import android.Manifest;
2  import android.content.pm.PackageManager;

```

```
3  import android.database.Cursor;
4  import android.os.Bundle;
5  import android.telephony.SmsManager;
6  import android.provider.Telephony;
7  import android.view.View;
8  import android.widget.AdapterView;
9  import android.widget.Button;
10 import android.widget.EditText;
11 import android.widget.ListView;
12 import android.widget.Toast;
13
14 import androidx.annotation.NonNull;
15 import androidx.appcompat.app.AppCompatActivity;
16 import androidx.core.app.ActivityCompat;
17 import androidx.core.content.ContextCompat;
18
19 import java.util.ArrayList;
20 import java.util.List;
21
22 public class SmsActivity extends AppCompatActivity {
23
24     private static final int REQUEST_SMS_PERMISSIONS = 2002;
25     private Button btnReadSms, btnSendSms;
26     private ListView lvSms;
27     private EditText etPhone, etMessage;
28     private List<String> smsList = new ArrayList<>();
29     private ArrayAdapter<String> adapter;
30
31     @Override
32     protected void onCreate(Bundle savedInstanceState) {
33         super.onCreate(savedInstanceState);
34         setContentView(R.layout.activity_sms);
35
36         btnReadSms = findViewById(R.id.btn_read_sms);
37         btnSendSms = findViewById(R.id.btn_send_sms);
38         lvSms = findViewById(R.id.lv_sms);
39         etPhone = findViewById(R.id.et_phone);
40         etMessage = findViewById(R.id.et_message);
41
42         adapter = new ArrayAdapter<>(this,
43 android.R.layout.simple_list_item_1, smsList);
44         lvSms.setAdapter(adapter);
45
46         btnReadSms.setOnClickListener(v -> checkSmsPermission());
47         btnSendSms.setOnClickListener(v -> sendSms());
48     }
```

```

49     // 检查短信权限 (读取+发送)
50     private void checkSmsPermission() {
51         String[] permissions = {Manifest.permission.READ_SMS,
Manifest.permission.SEND_SMS};
52         if (ContextCompat.checkSelfPermission(this, permissions[0]) !=
PackageManager.PERMISSION_GRANTED
53             || ContextCompat.checkSelfPermission(this, permissions[1]) !=
PackageManager.PERMISSION_GRANTED) {
54             ActivityCompat.requestPermissions(this, permissions,
REQUEST_SMS_PERMISSIONS);
55         } else {
56             readSms();
57         }
58     }
59
60     // 读取短信
61     private void readSms() {
62         smsList.clear();
63         Cursor cursor = getContentResolver().query(
64             Telephony.Sms.CONTENT_URI,
65             null, null, null, null
66         );
67
68         if (cursor != null) {
69             while (cursor.moveToNext()) {
70                 String address =
cursor.getString(cursor.getColumnIndex(Telephony.Sms.ADDRESS)); // 发件人号码
71                 String body =
cursor.getString(cursor.getColumnIndex(Telephony.Sms.BODY)); // 短信内容
72                 smsList.add("发件人: " + address + "\n内容: " + body);
73             }
74             adapter.notifyDataSetChanged();
75             cursor.close();
76         }
77     }
78
79     // 发送短信
80     private void sendSms() {

```

4. 关键细节

- **短信表字段：** ADDRESS（发件人号码）、 BODY（短信内容）、 DATE（时间）；
- **SmsManager 分段发送：**长短信需用 `sendMultipartTextMessage` 方法；
- **安卓 12+ 适配：**发送短信权限无需额外适配，但需注意用户隐私政策声明。

三、处理彩信（MMS）

彩信（MMS）包含图片、音频等多媒体内容，读取彩信需访问 `Telephony.Mms` 和 `Telephony.Mms.Part` 表，实现复杂度较高。

1. 权限申请

Code block

```
1 <uses-permission android:name="android.permission.READ_SMS" /> <!-- 彩信属于短信
   扩展，需此权限 -->
2 <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" /> <!--
   - 读取彩信附件 -->
```

2. 核心逻辑（读取彩信）

Code block

```
1 // 读取彩信示例（简化版）
2 private void readMms() {
3     mmsList.clear();
4     // 查询彩信列表
5     Cursor mmsCursor = getContentResolver().query(
6         Telephony.Mms.CONTENT_URI,
7         new String[]{Telephony.Mms._ID, Telephony.Mms.SUBJECT,
8             Telephony.Mms.DATE},
9         null, null, null
10    );
11    if (mmsCursor != null) {
12        while (mmsCursor.moveToNext()) {
13            String id =
14                mmsCursor.getString(mmsCursor.getColumnIndex(Telephony.Mms._ID));
15            String subject =
16                mmsCursor.getString(mmsCursor.getColumnIndex(Telephony.Mms.SUBJECT));
17            // 查询彩信附件（图片/音频）
18            Cursor partCursor = getContentResolver().query(
19                Telephony.Mms.Part.CONTENT_URI,
20                new String[]{Telephony.Mms.Part.FILENAME,
21                    Telephony.Mms.Part.CONTENT_TYPE},
22                Telephony.Mms.Part.MSG_ID + " = ?",
23                new String[]{id},
24                null
25            );
```



```

24         if (partCursor != null) {
25             while (partCursor.moveToNext()) {
26                 String contentType =
partCursor.getString(partCursor.getColumnIndex(Telephony.Mms.Part.CONTENT_TYPE)
);
27                 if (contentType.startsWith("image/")) { // 筛选图片附件
28                     mmsList.add("彩信主题: " + subject + "\n类型: 图片");
29                 }
30             }
31             partCursor.close();
32         }
33     }
34     mmsCursor.close();
35     adapter.notifyDataSetChanged();
36 }
37 }

```

3. 注意事项

- 彩信数据分散在 `Mms` 和 `Mms.Part` 表，需通过 `MSG_ID` 关联查询；
- 彩信附件存储路径需通过 `CONTENT_LOCATION` 字段获取；
- 安卓 10+ 需使用 `MediaStore` 访问彩信附件，避免直接操作文件路径。

四、从相册获取图片

1. 权限与 Intent 配置

- 安卓 10+：无需存储权限，直接通过 `MediaStore` 或 `ACTION_PICK` 访问相册；
- 安卓 9 及以下：需 `READ_EXTERNAL_STORAGE` 权限。

2. 核心 Intent: ACTION_PICK

通过系统相册的 `ACTION_PICK` Intent 选择图片，返回图片 URI。

3. 实战实现

步骤 1: XML 布局

Code block

```

1  <!-- activity_album.xml -->
2  <?xml version="1.0" encoding="utf-8"?>
3  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"

```

```

6      android:orientation="vertical"
7      android:gravity="center">
8
9      <Button
10         android:id="@+id/btn_pick_photo"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="从相册选择图片" />
14
15      <ImageView
16         android:id="@+id/iv_photo"
17         android:layout_width="300dp"
18         android:layout_height="300dp"
19         android:scaleType="centerCrop"
20         android:layout_marginTop="20dp" />
21
22  </LinearLayout>

```

步骤 2: Java 逻辑

Code block

```

1  import android.Manifest;
2  import android.content.Intent;
3  import android.content.pm.PackageManager;
4  import android.net.Uri;
5  import android.os.Build;
6  import android.os.Bundle;
7  import android.provider.MediaStore;
8  import android.view.View;
9  import android.widget.Button;
10 import android.widget.ImageView;
11 import android.widget.Toast;
12
13 import androidx.annotation.NonNull;
14 import androidx.annotation.Nullable;
15 import androidx.appcompat.app.AppCompatActivity;
16 import androidx.core.app.ActivityCompat;
17 import androidx.core.content.ContextCompat;
18 import androidx.core.content.FileProvider;
19
20 import com.bumptech.glide.Glide;
21
22 import java.io.File;
23
24 public class AlbumActivity extends AppCompatActivity {

```

```

25
26     private static final int REQUEST_PICK_PHOTO = 3001;
27     private static final int REQUEST_STORAGE_PERMISSION = 3002;
28     private Button btnPickPhoto;
29     private ImageView ivPhoto;
30
31     @Override
32     protected void onCreate(Bundle savedInstanceState) {
33         super.onCreate(savedInstanceState);
34         setContentView(R.layout.activity_album);
35
36         btnPickPhoto = findViewById(R.id.btn_pick_photo);
37         ivPhoto = findViewById(R.id.iv_photo);
38
39         btnPickPhoto.setOnClickListener(v -> pickPhotoFromAlbum());
40     }
41
42     // 从相册选择图片
43     private void pickPhotoFromAlbum() {
44         // 安卓9及以下检查存储权限
45         if (Build.VERSION.SDK_INT < Build.VERSION_CODES.Q &&
46             ContextCompat.checkSelfPermission(this,
Manifest.permission.READ_EXTERNAL_STORAGE) !=
PackageManager.PERMISSION_GRANTED) {
47             ActivityCompat.requestPermissions(this, new String[]
{Manifest.permission.READ_EXTERNAL_STORAGE}, REQUEST_STORAGE_PERMISSION);
48             return;
49         }
50
51         // 启动系统相册
52         Intent intent = new Intent(Intent.ACTION_PICK,
MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
53         startActivityForResult(intent, REQUEST_PICK_PHOTO);
54     }
55
56     // 处理相册返回结果
57     @Override
58     protected void onActivityResult(int requestCode, int resultCode, @Nullable
Intent data) {
59         super.onActivityResult(requestCode, resultCode, data);
60         if (requestCode == REQUEST_PICK_PHOTO && resultCode == RESULT_OK &&
data != null) {
61             Uri photoUri = data.getData(); // 获取选中图片的URI
62             // 使用Glide加载图片 (需添加依赖: implementation
'com.github.bumptech.glide:glide:4.15.1')
63             Glide.with(this).load(photoUri).into(ivPhoto);
64         }

```

```
65     }
66
67     @Override
68     public void onRequestPermissionsResult(int requestCode, @NonNull String[]
permissions, @NonNull int[] grantResults) {
69         super.onRequestPermissionsResult(requestCode, permissions,
grantResults);
70         if (requestCode == REQUEST_STORAGE_PERMISSION) {
71             if (grantResults.length > 0 && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
72                 pickPhotoFromAlbum();
73             } else {
74                 Toast.makeText(this, "存储权限被拒绝, 无法访问相册",
Toast.LENGTH_SHORT).show();
75             }
76         }
77     }
```

4. 关键细节

- **Glide 依赖**: 需在 `build.gradle` 添加 Glide 库以简化图片加载;
- **URI 适配**: 安卓 7.0+ 需用 `FileProvider` 处理文件 URI, 避免 `FileUriExposedException`;
- **安卓 10+ 分区存储**: 无需存储权限, 直接通过 `MediaStore` 访问图片。

五、通用注意事项

1. 权限动态申请

所有危险权限（联系人、短信、存储）均需在运行时申请, 遵循 Android 6.0+ 权限机制。

2. 隐私政策声明

Google Play 要求访问用户隐私数据（联系人、短信）的 App 必须在隐私政策中说明数据用途, 否则可能被下架。

3. 兼容性适配

- 安卓 10+ 分区存储: 避免直接操作文件路径, 使用 `MediaStore`;
- 安卓 13+ 权限分组: 部分权限（如通知、照片）申请逻辑有变化。

4. 性能优化

- 查询大量数据（如联系人、短信）时, 建议在子线程执行, 避免阻塞主线程;
- 使用 `CursorLoader` 替代直接查询 `ContentResolver`, 自动处理生命周期。

通过以上实现，你可以完成联系人、短信、彩信和相册图片的核心操作。每个功能均需注意权限处理和系统版本适配，确保 App 在不同设备上稳定运行！

（注：文档部分内容可能由 AI 生成）